Designing a Modular Pipeline for Probing Linguistic Representations in Large Language Models

Lars Heijnen
Neurocomputation of Language Research Group
800802-B-6: Research Labs/Internships
Dr. Noortje J. Venhuizen & Dr. Harm Brouwer
Internship start date: 05-02-2025
Internship end date: 31-05-2025
Date of submission: 21-02-2025

Introduction & Context

During my internship, I will be a member of the Neurocomputation of Language research group and actively collaborate with them (*Neurocomputation of Language | Tilburg University*, n.d.). This group studies how language is processed, represented, and generated in the brain using computational modeling, neuroscience, and artificial intelligence. The performed work spans multiple disciplines, including linguistics, neuroscience, cognitive science, artificial intelligence, and machine learning.

The task given to me is to set up a pipeline to perform various probing tasks on Large Language Models (LLMs). Probing tasks on{Citation} LLMs analyze their internal representations to uncover how they encode linguistic and conceptual knowledge. Probing tasks examine whether LLMs capture linguistic, syntactic, or semantic information within their hidden states or embeddings, shedding light on the depth of their understanding and underlying mechanisms (Ju et al., 2024; Vulić et al., 2020). Since LLMs are often treated as black boxes (*The Black Box Problem*, 2023; *What Is Black Box AI and How Does It Work?*, 2024), probing provides valuable insights into why a model makes certain predictions. Probing helps researchers identify how many models rely on surface-level statistical patterns in contrast to deeper language structures.

A paper by Tenney et al. (2019) introduces a variety of probing tasks. They determine which aspects of are encoded. Their research directly relates to addressing the black box issue, by assessing the "depth of understanding" within LLMs. Furthermore, Shi et al. (2016) investigate whether the encoder in neural machine translation systems captures syntactic structure. Using logistic regression on the encoder's hidden states, the paper demonstrates that significant syntactic information is captured, supporting the idea that probing reveals the underlying syntactic and semantic signals behind model predictions. Moreover, Conneau et al. (2018) assess the linguistic properties of sentence embeddings by introducing ten different probing tasks, from surface features to deeper syntactic and semantic aspects. They show that even simple aggregation methods can capture significant language information, implying that sentence embeddings inherently retain structural details. Another paper by Belinkov (2022) examines probing classifiers, warning that high probing accuracy may arise from probe memorization rather than from genuine usage of encoded features by the original model. Probe

memorization can be defined as the probe learning to simply memorize patterns or superficial cues present in the training data rather than understanding deeper relationships. Belinkov advocates for more rigorous baselines and control measures to ensure reliable insights

The mentioned papers provide an important starting point for this internship.

Relevant Background and Coursework

Throughout my bachelor's degree, I've completed numerous courses that have collectively built a strong foundation of skills relevant to this internship. While many have contributed to my development, I'll highlight a few that stand out for their direct applicability. Introduction to Machine Learning gave me a solid understanding of core concepts, along with hands-on experience using Python libraries like scikit-learn for model building, evaluation, and tuning. In Data Structures and Algorithms, I learned to work with data structures such as dictionaries, sets, files, and basic data types. The course also explored algorithms like searching, sorting, and linked lists, as well as the basics of algorithmic complexity, which improved my logical thinking and problem-solving skills. Additionally, the course introduced key programming concepts, including functions, recursion, and object-oriented principles like inheritance, UML, and operator overloading. Finally, Language and AI at Eindhoven University of Technology enhanced my ability to process and analyze natural language data, deepening my understanding of semantic meaning and syntax, and teaching me how to effectively train models on linguistic data.

Goal

As mentioned, the goal is to develop a pipeline to perform different probing tasks on LLMs. The pipeline should be designed with a modular structure. This means that the pipeline is composed of separate, independent components that together form a unified system. This will allow for straightforward exchange, addition, removal, or modification of building blocks without interfering with the functionality of the entire system. This will be implemented in a Jupyter Notebook (*Project Jupyter Documentation — Jupyter Documentation 4.1.1 Alpha Documentation*, n.d.).

Before I start developing the pipeline, it is important to carefully consider my approach and allocate time for this thought process. In any case, the pipeline must work efficiently and be

useful in performing probing tasks. The goal of this pipeline is to apply novel probing tasks to LLMs that focus deep semantic information. As part of my internship, the pipeline will be tested by replicating previous findings, as well as by developing novel probing tasks, in collaboration with the NCL research group. Adding to this, not only must the pipeline operate efficiently, but it should also be user-friendly for others, therefore, a comprehensive README is essential.

Method & Approach

I began by reviewing literature related to my task, as I believe it is essential for gaining foundational knowledge in the domain in which I will be working. While I actively seek out relevant literature myself, I also receive valuable literature and information from the Neurocomputation of Language research group, such as the previously mentioned papers by Tenney et al. (2019) and Shi et al. (2016). Before writing this proposal, I have already been tasked to develop a logistic regression classifier to predict whether a word is a verb. The Jupyter Notebook I developed trains the model using BERT word embeddings to extract features from words in the Treebank dataset and evaluates its performance using accuracy, precision, recall, F1-score, and cross-validation. The results were promising.

As mentioned earlier, the pipeline should have a modular structure. Key components include loading the data, along with the necessary modules and libraries, followed by preprocessing, generating sentence embeddings, training models, and eventually evaluating model performance. To implement the pipeline, a wide variety of tools and frameworks must be used. Python will be the chosen programming language (*The Python Language Reference*, n.d.). For deep learning tasks, PyTorch is essential. PyTorch enables the loading and fine-tuning of neural network models, managing embeddings, and implementing custom probing classifiers (*PyTorch Documentation — PyTorch 2.6 Documentation*, n.d.). Sentence Transformers, built on Hugging Face Transformers, will be used to generate embeddings (*SentenceTransformers Documentation — Sentence Transformers Documentation*, n.d.). To further improve NLP capabilities, NLTK will be integrated to provide additional linguistic processing tools, including semantic analysis (*NLTK : Natural Language Toolkit*, n.d.). Data handling and preprocessing will mainly rely on Pandas (*Pandas Documentation — Pandas 2.2.3 Documentation*, n.d.) and NumPy (*NumPy Documentation*, n.d.), with Pandas structuring datasets and NumPy supporting mathematical operations. Finally, scikit-learn will be employed for building and evaluating

machine learning models for probing, offering various algorithms for model selection and performance evaluation (*Documentation Scikit-Learn: Machine Learning in Python — Scikit-Learn 0.21.3 Documentation*, n.d.).

To conclude, I am responsible for developing the pipeline, while other members of the Neurocomputation of Language research group focus on designing the probing tasks.

Relevance & Expected Output

Developing a pipeline for probing tasks on LLMs is highly relevant to advancing research in computational linguistics, cognitive neuroscience, and artificial intelligence. It is important to understand whether models rely on surface-level statistical patterns or deeper language structures to generate meaningful and contextually accurate responses, as this helps address the 'black box' problem in AI.

Planning

At the time of writing this proposal, I have already dedicated a total of 29.5 hours to preliminary activities. Specifically, I attended two Neurocomputation Lab Meetings (2 hours each, totaling 4 hours), met with Rémy from the lab for 1.5 hours, and read approximately three research papers in depth (6 hours). Additionally, the first task assigned by Dr. Harm Brouwer and Dr. Noortje J. Venhuizen took around 10 hours, and preparing this proposal required about 8 hours.

This leaves 138.5 hours remaining for the internship. With the internship scheduled from week 9 to the end of week 22, a span of 14 weeks, this translates to roughly 10 hours per week. Below is a general timeline of the internship.

Milestone	**Description**	**Internship Weeks**
Literature review	Gather and review relevant research and background materials.	Weeks 9-10
Pipeline Design & Planning	Define pipeline architecture and select necessary tools.	Weeks 11

Development	Implement data loading, preprocessing, and embedding generation modules	Weeks 12-14
Probing Task Implementation	Develop and integrate probing classifiers and evaluation components.	Weeks 15-18
Pipeline Testing & Validation	Test pipeline by replicating previous findings and refining novel probing tasks.	Weeks 19-20
Final Report Writing	Preparing comprehensive final report, including README and analysis of outcomes.	Weeks 21-22

References

Belinkov, Y. (2022). Probing Classifiers: Promises, Shortcomings, and Advances. *Computational Linguistics*, *48*(1), 207–219. https://doi.org/10.1162/coli_a_00422

Conneau, A., Kruszewski, G., Lample, G., Barrault, L., & Baroni, M. (2018). What you can cram into a single $&!#* vector: Probing sentence embeddings for linguistic properties. In I. Gurevych & Y. Miyao (Eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 2126–2136). Association for Computational Linguistics. https://doi.org/10.18653/v1/P18-1198

Documentation scikit-learn: Machine learning in Python—Scikit-learn 0.21.3 documentation. (n.d.). Retrieved February 20, 2025, from https://scikit-learn.org/0.21/documentation.html

Ju, T., Sun, W., Du, W., Yuan, X., Ren, Z., & Liu, G. (2024). *How Large Language Models Encode Context Knowledge? A Layer-Wise Probing Study* (arXiv:2402.16061). arXiv. https://doi.org/10.48550/arXiv.2402.16061

Neurocomputation of Language | Tilburg University. (n.d.). Retrieved February 21, 2025, from https://www.tilburguniversity.edu/about/schools/tshd/departments/dca/lab/neurocomputation-language

NLTK :: Natural Language Toolkit. (n.d.). Retrieved February 20, 2025, from https://www.nltk.org/index.html

NumPy Documentation. (n.d.). Retrieved February 20, 2025, from https://numpy.org/doc/

pandas documentation—Pandas 2.2.3 documentation. (n.d.). Retrieved February 20, 2025, from https://pandas.pydata.org/docs/

Project Jupyter Documentation—Jupyter Documentation 4.1.1 alpha documentation. (n.d.). Retrieved February 20, 2025, from https://docs.jupyter.org/en/latest/

PyTorch documentation—PyTorch 2.6 documentation. (n.d.). Retrieved February 20, 2025, from https://pytorch.org/docs/stable/index.html

SentenceTransformers Documentation—Sentence Transformers documentation. (n.d.). Retrieved February 20, 2025, from https://sbert.net/

Shi, X., Padhi, I., & Knight, K. (2016). Does String-Based Neural MT Learn Source Syntax? In J. Su, K. Duh, & X. Carreras (Eds.), *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing* (pp. 1526–1534). Association for Computational Linguistics. https://doi.org/10.18653/v1/D16-1159

Tenney, I., Xia, P., Chen, B., Wang, A., Poliak, A., McCoy, R. T., Kim, N., Durme, B. V., Bowman, S. R., Das, D., & Pavlick, E. (2019). *What do you learn from context? Probing for sentence structure in contextualized word representations* (arXiv:1905.06316). arXiv. https://doi.org/10.48550/arXiv.1905.06316

The Black Box Problem: Opaque Inner Workings of Large Language Models. (2023, October 23). Prompt Engineering. https://promptengineering.org/the-black-box-problem-opaque-inner-workings-of-large-language-models/

The Python Language Reference. (n.d.). Python Documentation. Retrieved February 20, 2025, from https://docs.python.org/3/reference/index.html

Vulić, I., Ponti, E. M., Litschko, R., Glavaš, G., & Korhonen, A. (2020). Probing Pretrained Language Models for Lexical Semantics. In B. Webber, T. Cohn, Y. He, & Y. Liu (Eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 7222–7240). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.586

What Is Black Box AI and How Does It Work? | IBM. (2024, October 29). https://www.ibm.com/think/topics/black-box-ai